AZ AI ÚTVONALTERVEZŐ ALGORITMUS

AZ ALGORITMUS ALAPELVEI:

1. A rover csak olyan célt választ, amelyhez biztonságosan oda tud menni, ott szükség esetén tud bányászni, és utána vissza is tud térni.
2. A rover minden lépésnél figyelembe veszi:
    - az akkumulátort
    - nappal/éjszaka ciklust
    - a hátralévő időt
    - a hazajutás lehetőségét (ez a legfontosabb cél a programomban)
    - a kibányaszható érmeket
3. A rover nemcsak a pillanatnyi legjobb műveletet nézi, hanem előre ellenőrzi, hogy a teljes terv végrehajtható-e.

Ezért a rendszer nem egyszerűen útkereső algoritmus (A* ebben az esetben), hanem egy biztonságos tervellenőrző és döntéshozó logika

A DÖNTÉSI RENDSZER: A kódomban a következő funkciókkal találkozhatnak

1. Hazatérési távolság előszámítása
    - tavolsagHazatolBFS
2. Útvonal keresése két pont között (A* algoritmus)
    - astarUtvonalKereses
3. Minimális szükséges idő becslése egy teljes útra (utesek = (óra)ütések)
    - teljesUtMinUtesek
4. Egy teljes terv szimulációja és megvalósíthatóságának ellenőrzése
    - utvonalTervSzimulacio
5. Biztonságos célpont kiválasztása
    - pick_target_safe
6. Biztonságos sebesség kiválasztása az aktuális körben (minimalizáltam a 3. sebesség használatát)
    - choose_safe_move_speed
7. Az aktuális kör tényleges végrehajtása
    - sim_step

MAGAS SZINTŰ AI STRATÉGIA

A rover minden körben a következőket próbálja végrehajtani:

1. Ha éppen egy ásványon áll:
    ellenőrizze, hogy most biztonságos-e bányászni (van-e elég akkumulátora, haza tud-e jutni):
    →ha igen, bányásszon
2. Ha nem bányászik:
    keressen egy biztonságosan vállalható ásványcélt
3. Ha van ilyen cél:

keressen hozzá útvonalat
válasszon olyan sebességet, amely után a teljes terv még mindig biztonságos

4. Ha nincs biztonságos ásványcél, vagy oda most nem lehet biztonságosan elindulni:
próbáljon hazafelé menni (pl: az összes többi ásvány túl messze van és a hazatérés kockázatban van)

5. Ha a hazafelé mozgás sem lehetséges biztonságosan:
várakozzon (nincs elegendő energia a mozgáshoz)

PSZEUDOKÓD: TELJES AI DÖNTÉSI FOLYAMAT

Eljárás: RoverAI_KoronkentiDontes (valós megfelelő: sim_step)
Bemenet:

- rover aktuális állapota
- küldetés adatai
- térkép
- hazatérési távolságok
- log fájlok

Kimenet:

- kör végrehajtása
- rover állapotának frissítése
- naplózás (adatok betöltése a fájlba, amit majd a GUIhoz használok fel)

Pszeudokód:

```
HA a rover ásvány mezőn áll
    HA a bányászás megengedett (banyaszasLehetseges / can_do_mine) AKKOR
        számold ki a bányászás utáni akkut (banyaszasAlkalmazasa / apply_mine)
        ellenőrizd, hogy a bányászás után még biztosan haza lehet-e jutni
        (utvonalTervSzimulacio)
            HA igen AKKOR
                végezd el a bányászatot
                frissítsd az ásványdarabszámot
                töröld az ásványt a térképről
                frissítsd az akkut
                naplózd a DIGGING műveletet
                lépj a következő körre
                VÉGE
    VÉGE
VÉGE
keress biztonságos ásványcélt (pick_target_safe)
HA van biztonságos cél AKKOR
    keress pontos útvonalat a célhoz (astarUtvonalKereses)
    HA van útvonal AKKOR
        számold ki, hány lépés van még a célig
        válassz biztonságos sebességet (choose_safe_move_speed)
        HA létezik megfelelő sebesség AKKOR
            mozgasd a rovert az útvonal megfelelő pontjára
            frissítsd a megtett cellák számát
            frissítsd az akkut
```

```
                naplózd a MOVING műveletet
                lépj a következő körre
                VÉGE
            VÉGE
        VÉGE
        HA a rover nincs otthon AKKOR
            keress útvonalat haza (astarUtvonalKereses)
            HA van hazafelé útvonal AKKOR
                válassz biztonságos hazafelé sebességet (choose_safe_move_speed)
                    HA létezik megfelelő sebesség AKKOR
                    mozgasd a rovert hazafelé
                    frissítsd a megtett cellák számát
                    frissítsd az akkut
                    naplózd a MOVING műveletet
                    lépj a következő körre
                    VÉGE
            VÉGE
        VÉGE
```

A TERV ELLENŐRZÉSE

A rover intelligenciája nem az A* útkeresés brute force-ra alakul, hanem az, hogy egy tervet előre le tud ellenőrizni. Ez a következő függvényben történik:

utvonalTervSzimulacio

(eredetileg: simulate_trip_plan)

A függvény azt vizsgálja meg, hogy egy terv végrehajtható-e:

- eljutás a célhoz
- bányászás
- visszatérés

A terv szimulációja során minden körben több akciójelöltet vizsgál:

- várakozás
- mozgás 1-es sebességgel
- mozgás 2-es sebességgel
- mozgás 3-as sebességgel
- bányászás

Majd kiválasztja a legjobb műveletet.

TERV SZIMULÁCIÓ

Eljárás: utvonalTervSzimulacio / simulate_trip_plan

Bemenet:

- kezdő idő
- kezdő akkumulátor
- hátralévő odafelé lépések
- kell-e még bányászni
- hátralévő visszaút

- küldetés végső időkorlátja

Kimenet:

- végrehajtható-e a terv
- meddig jut a szimuláció

mennyi akku marad

Pszeudokód:

állítsd be az aktuális időt a kezdő időre
állítsd be az aktuális akkut a kezdő akku értékére
AMÍG az aktuális idő kisebb, mint a maximális idő
    HA már nincs odafelé út ÉS nem kell bányászni ÉS nincs visszaút AKKOR
        Térj vissza sikerrel
        VÉGE
    állapítsd meg, hogy nappal van-e
    (nap_van_ciklus)
számold ki, hogy a következő művelet után mennyi idő marad
kezdetben nincs kiválasztott művelet, a legjobb maradó akku nagyon kicsi (-1e100), a legjobb haladás értéke érvénytelen (-1)

Alfunkciók az útvonaltervezéshez +a pszeudokódjuk

<u>1. VÁRAKOZÁS VIZSGÁLATA</u>

HA a várakozás szabályos (varakozasLehetseges) AKKOR
        számold ki a várakozás utáni akkut (varakozasAlkalmazasa)
        HA a maradék terv minimális időigénye belefér a hátralévő időbe
        (teljesUtMinUtesek / trip_min_ticks) AKKOR
            jelöld ezt lehetséges műveletnek (van művelet, új akkumulátor értéke, haladás)
            VÉGE
        VÉGE

<u>2. MOZGÁS VIZSGÁLATA</u>

HA még van odafelé út VAGY már csak a visszaút maradt AKKOR
    határozd meg, hogy a jelenlegi szakaszban hány lépés maradt
    CIKLUS sebesség = 1..3
        HA a sebesség nagyobb, mint a megtehető lépések
            ugord át
            VÉGE
        HA a mozgás nem szabályos (mozgasLehetseges / can_do_move) AKKOR
            ugord át
            VÉGE
        csökkentsd a megfelelő szakasz hátralévő lépésszámát
        számold ki az új akkut (mozgasAlkalmazasa / apply_move)
        HA a maradék terv minimális ideje belefér (teljesUtMinUtesek) AKKOR

hasonlítsd össze ezt a műveletet a jelenlegi legjobbal
előnyben részesítve:
1. nagyobb maradó akkut
2. holtverseny esetén nagyobb haladást
VÉGE
CIKLUS VÉGE
VÉGE
3. BÁNYÁSZÁS VIZSGÁLATA
HA már odaértünk ÉS még kell bányászni AKKOR
HA a bányászás szabályos (banyaszasLehetseges) AKKOR
számold ki a bányászás utáni akkut (banyaszasAlkalmazasa)
HA a maradék terv minimális ideje belefér (teljesUtMinUtesek) AKKOR
hasonlítsd össze ezt a műveletet a jelenlegi legjobbal
VÉGE
VÉGE
VÉGE
VÉGE
HA nincs egyetlen érvényes művelet sem
térj vissza sikertelenséggel
VÉGE
alkalmazd a kiválasztott legjobb akciót
frissítsd:
- akkut
- hátralévő odaút lépéseit
- kell-e még bányászni
- hátralévő visszaút lépéseit
növeld az időt eggyel
CIKLUS VÉGE
ha lejárt az idő és a terv még nincs kész térj vissza sikertelenséggel

BIZTONSÁGOS CÉLPONT KIVÁLASZTÁSA

Eljárás: pick_target_safe
Bemenet:
- rover állapota
- küldetés állapota
- térkép
- hazatérési távolságok

Kimenet:
- van-e megfelelő cél
- ha igen, annak koordinátái

Pszeudokód:

kezdetben nincs legjobb célpont (-1)
járd be a teljes térképet
HA az adott cella nem ásvány
    ugord át
VÉGE
HA az adott cellából a bázis nem elérhető (home_dist (uint8_t) 255) AKKOR
    ugord át
VÉGE
keress pontos útvonalat a rover jelenlegi helyétől a célcelláig (astarUtvonalKereses)
HA nincs útvonal
    ugord át
VÉGE
számold ki:
    - odaút hosszát
    - hazautat
futtasd le a teljes terv szimulációját (utvonalTervSzimulacio)
HA a terv nem végrehajtható
    ugord át
VÉGE
hasonlítsd össze a célpontot a jelenlegi legjobbal
előnyben részesítve:
    1. rövidebb teljes oda-vissza utat
    2. azonos esetben rövidebb odautat
    3. azonos esetben nagyobb végső akkut
HA találtál megfelelő célpontot térj vissza annak koordinátáival
KÜLÖNBEN térj vissza sikertelenséggel

BIZTONSÁGOS SEBESSÉG KIVÁLASZTÁSA

Megjegyzés: 3. sebességet igyekeztem kerülni, mivel tapasztalatom szerint hosszabb utak során problémát okozott az akkumulátorral. Nappal is ha 3. sebességgel mozgunk akkor mínuszban maradunk, így meghoztam azt a döntést, hogy biztonság és hazatérés legyen az első helyen. Lásd a kivétel esetet.
Eljárás: choose_safe_move_v
Bemenet:
    - rover aktuális állapota
    - küldetés állapota
    - útvonal
    - hátralévő lépések száma
    - cél típusa
    - cél koordinátái
Kimenet:

- választott sebesség
- vagy 0, ha most nem szabad mozdulni

Pszeudokód:

állapítsd meg, hogy nap van-e
állíts fel egy alap sebességsorrendet
ha nappal van:
    preferencia: 2, 1, 3
különben:
    preferencia: 1, 2, 3
számold ki, hogy a maradék teljes terv minimálisan mennyi időt igényel (teljesUtMinUtesek)
ha nagyon szoros az idő
    akkor a sorrend legyen: 3, 2, 1
végigpróbálod a sebességeket a sorrend szerint, minden sebességnél:
HA a sebesség nagyobb, mint a hátralévő lépésszám
    ugord át
VÉGE
HA a mozgás nem szabályos
    (mozgasLehetseges / can_do_move)
AKKOR
    ugord át
VÉGE
számold ki a mozgás utáni akkut (mozgasAlkalmazasa / apply_move)
szimuláld a hátralévő teljes tervet (utvonalTervSzimulacio)
HA a terv így még végrehajtható
    térj vissza ezzel a sebességgel
VÉGE
ha egyik sebesség sem megfelelő
    térj vissza 0-val

MIÉRT NEM CSAK A* AZ AI?

Fontos hangsúlyozni, hogy az AI algoritmus nem azonos az A* útkereséssel.
Az A* csak azt mondja meg:
hogyan lehet eljutni két pont között.
Az AI viszont ennél sokkal többet csinál:

- kiválasztja, hogy érdemes-e egyáltalán odamenni
- ellenőrzi, hogy a célpont vállalható-e
- ellenőrzi, hogy a bányászás után is marad-e esély a hazatérésre
- kiválasztja a sebességet
- eldönti, hogy inkább várjon-e
- szükség esetén feladja az ásványgyűjtést és hazafelé indul

Ezért az AI magja valójában:

előrelátó tervellenőrzés + biztonságos döntéshozatal

AZ AI ALGORITMUS LÉNYEGE EGY MONDATBAN

A rover AI minden körben olyan műveletet választ, amely nemcsak az adott pillanatban szabályos, hanem előretekintve is biztosítja, hogy a teljes terv időben, elegendő energiával és biztonságos hazatéréssel végrehajtható maradjon.

RÖVID ÖSSZEFOGLALÁS

A rover AI algoritmusa négy fő szintből áll:

1. Útvonalinformáció előállítása
    - cellatolHazaTav (BFS)
    - astarUtvonalKereses
2. Idő- és energiaigény becslése
    - teljesUtMinUtesek
3. Jövőbeli terv megvalósíthatóságának ellenőrzése
    - utvonalTervSzimulacio
4. Konkrét döntés és végrehajtás
    - célpont kiválasztás
    - sebesség kiválasztás
    -sim_step

Ez együtt alkotja a rover tényleges AI viselkedését.